EWB COURSES

AWS ARCHITECT ASSOCIATE | STUDENT PROJECT REPORT

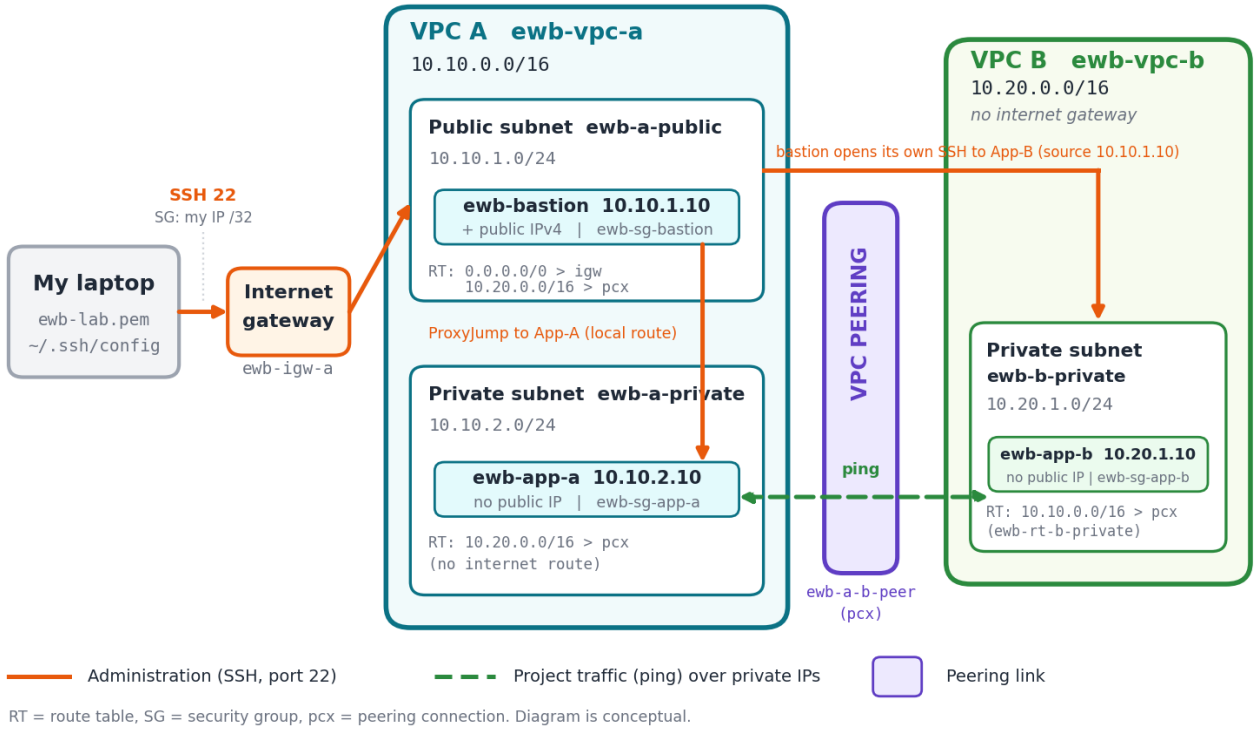
VPC Peering

Connect two VPCs. Prove private connectivity.

Student name	OM P. PATEL
Batch	Batch 1
Project title	AWS VPC Peering (Project 01)
AWS Region	Asia Pacific (Mumbai) / ap-south-1
Completion date	23 October 2026

1. Architecture and resource inventory

I built two VPCs with different IP ranges, joined them with VPC peering, and used one public bastion host in VPC A to reach the private servers.



Resource	Name tag	CIDR / private IP	Actual ID
VPC A	ewb-vpc-a	10.10.0.0/16	vpc-00c8d9347333fea0a
VPC B	ewb-vpc-b	10.20.0.0/16	vpc-0e01512ad16cf6466
Public subnet	ewb-a-public	10.10.1.0/24	subnet-00335739e3a3596ee
Private subnet A	ewb-a-private	10.10.2.0/24	subnet-00beebae7491ee4cd
Private subnet B	ewb-b-private	10.20.1.0/24	subnet-0e86b4c7653a82aee
Internet gateway	ewb-igw-a	VPC A only	igw-07c423e175deced49
Route table	ewb-rt-a-public	ewb-a-public	rtb-021ba85fc6dc01e75
Route table	ewb-rt-a-private	ewb-a-private	rtb-0cf6f0d67b4561f51
Route table	ewb-rt-b-private	ewb-b-private	rtb-07534f29f786d4688
Peering connection	ewb-a-b-peer	VPC A <> VPC B	pcx-052062980f03b6c17
Bastion instance	ewb-bastion	10.10.1.10 (+ public IP)	i-06b4083dd11bfdac8
App-A instance	ewb-app-a	10.10.2.10	i-09fe8703c989ef0b6
App-B instance	ewb-app-b	10.20.1.10	i-08f2488d596b89730

Bastion public IPv4: [REDACTED] **Instance type:** t3.micro (Amazon Linux 2023)

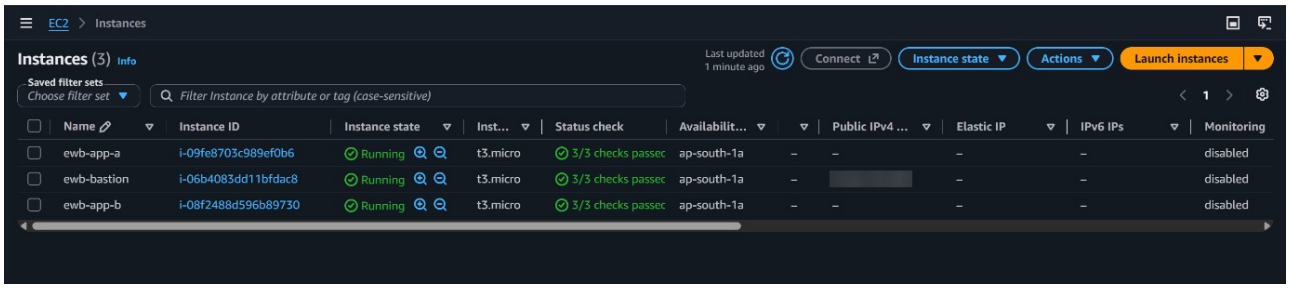


Figure 1. EC2 console: all three instances running with 3/3 status checks. Only the bastion has a public IPv4 address.

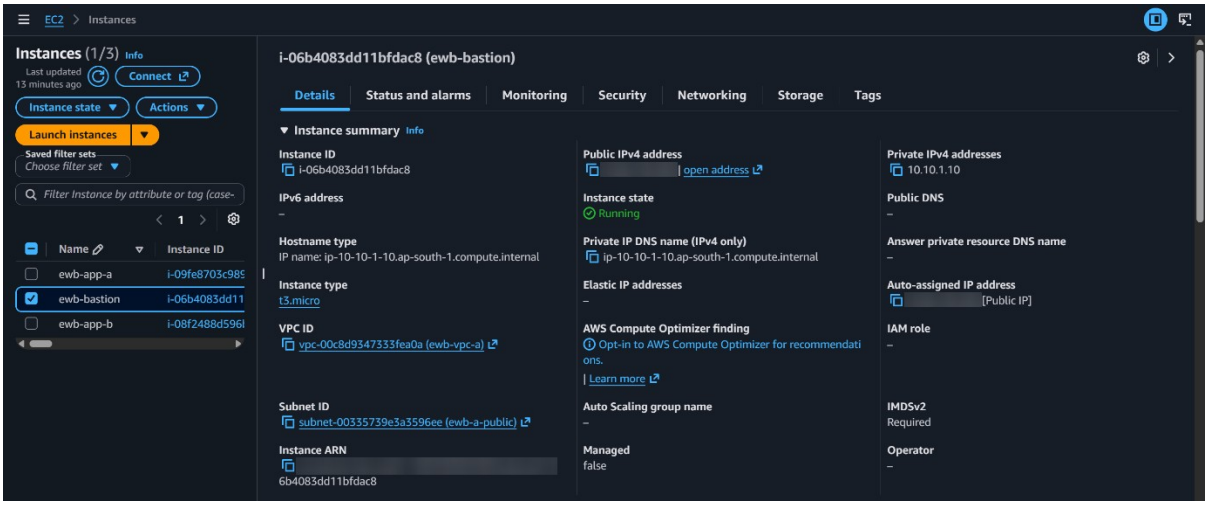


Figure 2. EC2 console, ewb-bastion details: private IP 10.10.1.10, public IPv4 [REDACTED], in ewb-vpc-a / ewb-a-public.

2. Subnets and route tables

Each subnet is explicitly associated with its own route table, and the internet gateway is attached to VPC A only.

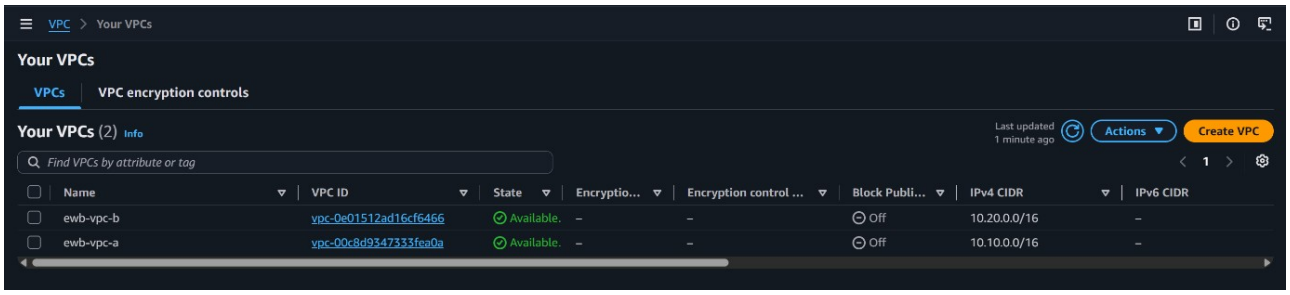


Figure 3. VPC console: ewb-vpc-a (10.10.0.0/16) and ewb-vpc-b (10.20.0.0/16), both Available.

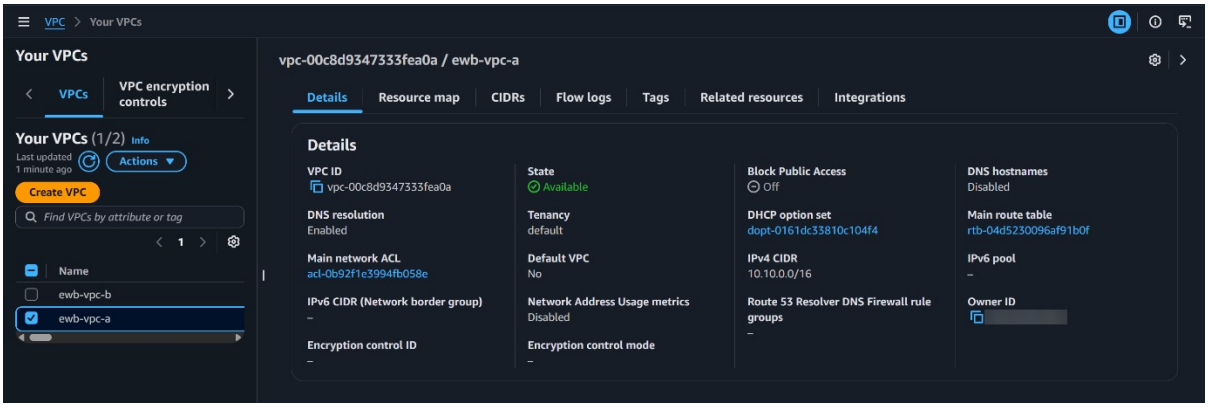


Figure 4. VPC details: ewb-vpc-a, vpc-00c8d9347333fea0a, IPv4 CIDR 10.10.0.0/16.

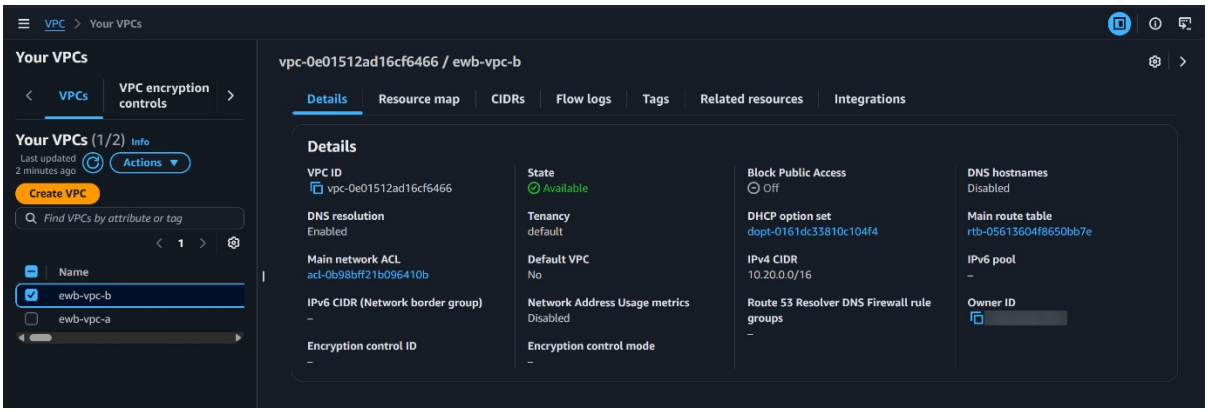


Figure 5. VPC details: ewb-vpc-b, vpc-0e01512ad16cf6466, IPv4 CIDR 10.20.0.0/16.

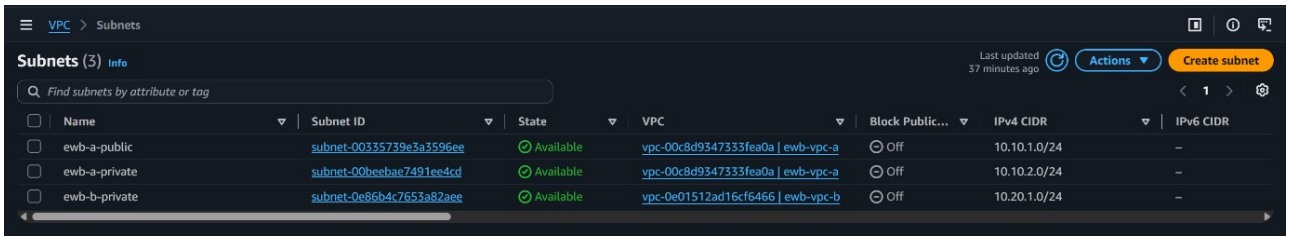


Figure 6. Subnets: ewb-a-public, ewb-a-private and ewb-b-private with their CIDRs and VPCs.

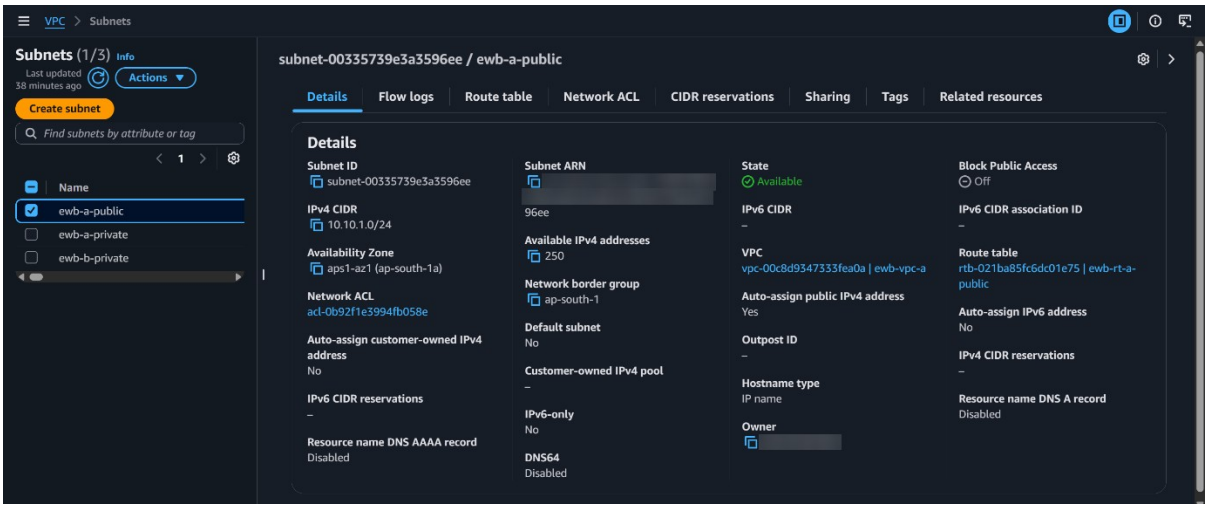


Figure 7. Subnet details: ewb-a-public, 10.10.1.0/24, ap-south-1a, route table ewb-rt-a-public, auto-assign public IPv4 enabled.

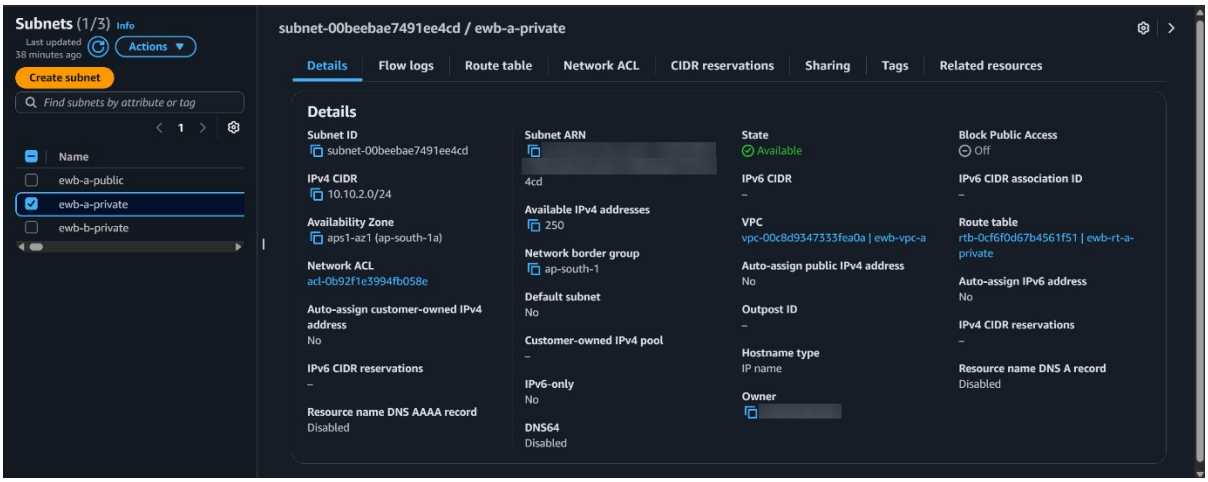


Figure 8. Subnet details: ewb-a-private, 10.10.2.0/24, ap-south-1a, route table ewb-rt-a-private, auto-assign public IPv4 disabled.

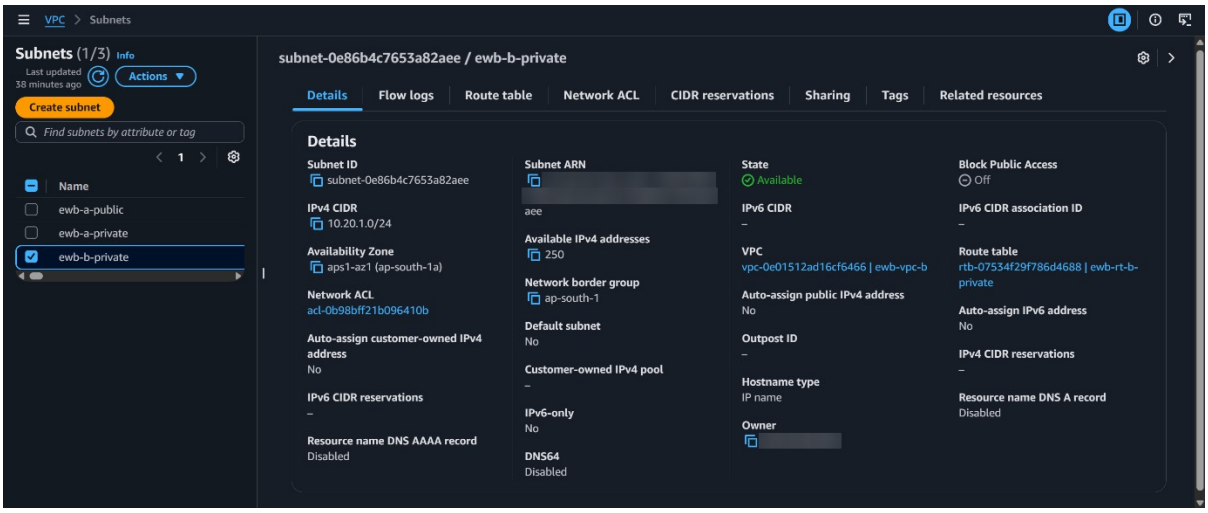


Figure 9. Subnet details: ewb-b-private, 10.20.1.0/24, ap-south-1a, route table ewb-rt-b-private, auto-assign public IPv4 disabled.

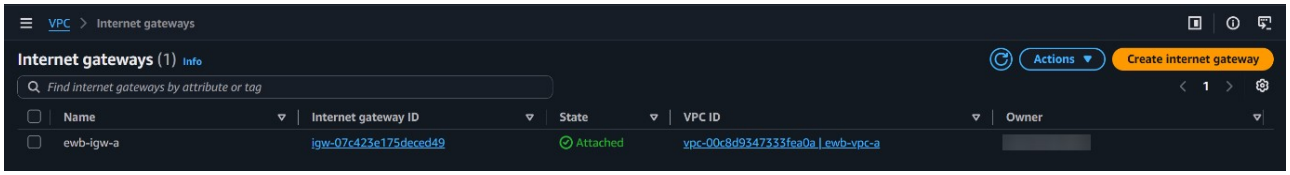


Figure 10. Internet gateway ewb-igw-a, state Attached to ewb-vpc-a.

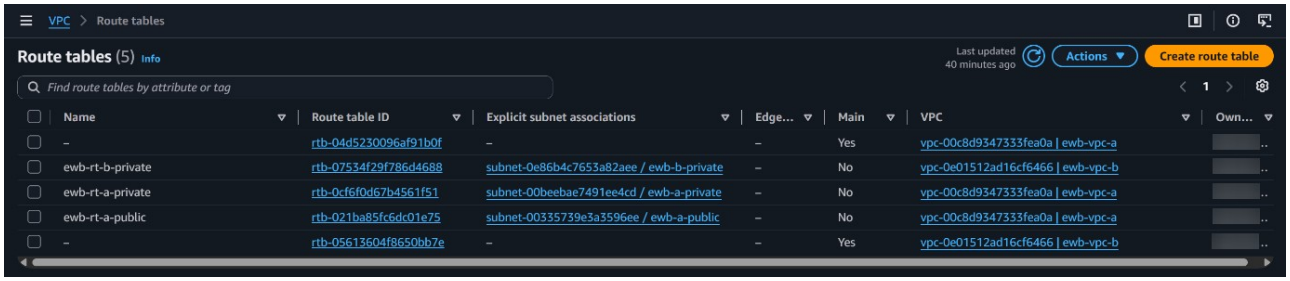


Figure 11. Route tables with their explicit subnet associations.

Route tables before peering (Checkpoint 01). The public route table has the internet route. The two private route tables only have their local route.

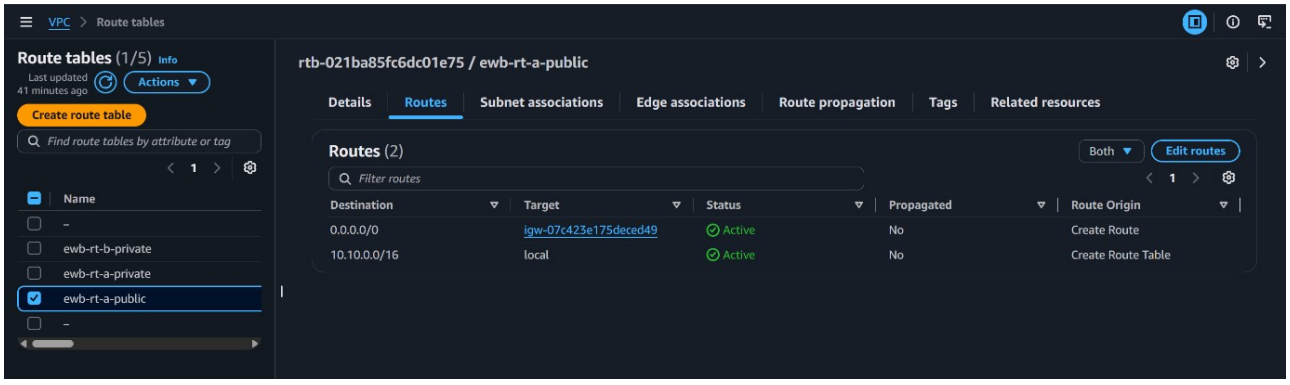


Figure 12. ewb-rt-a-public: 10.10.0.0/16 local and 0.0.0.0/0 to ewb-igw-a.

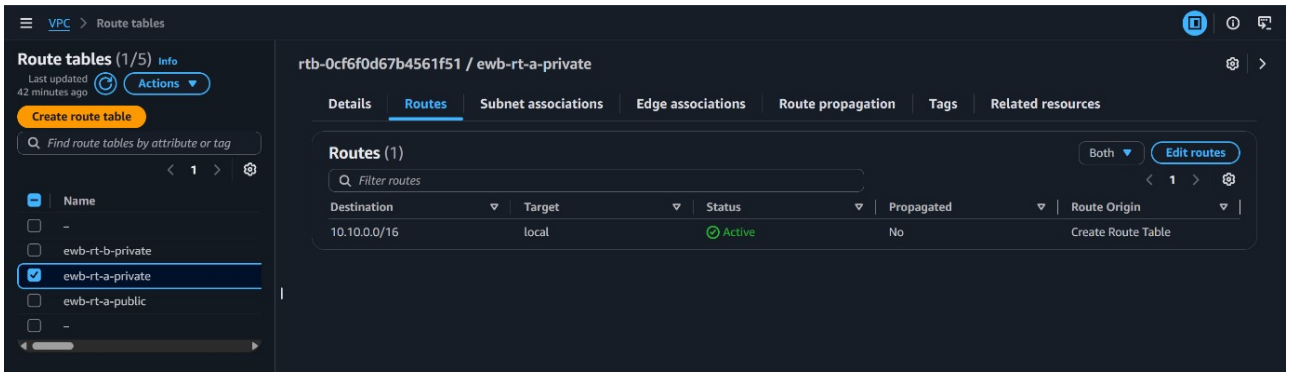


Figure 13. ewb-rt-a-private: local route only.

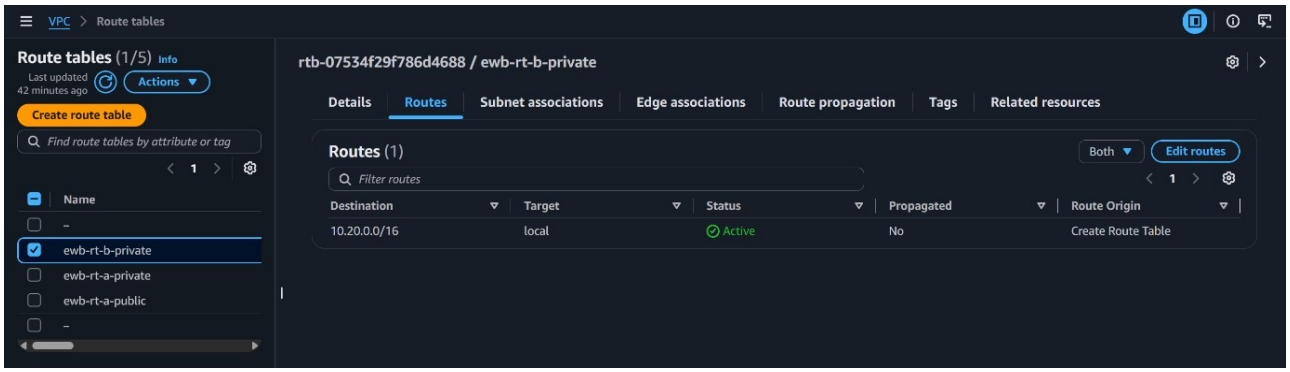


Figure 14. ewb-rt-b-private: local route only.

Route tables after peering (Checkpoint 04). After the peering became Active I added the peering routes on both sides:

Route table	Destination	Target
ewb-rt-a-public	10.20.0.0/16	pcx-052062980f03b6c17
ewb-rt-a-private	10.20.0.0/16	pcx-052062980f03b6c17
ewb-rt-b-private	10.10.0.0/16	pcx-052062980f03b6c17

3. Peering connection and security groups

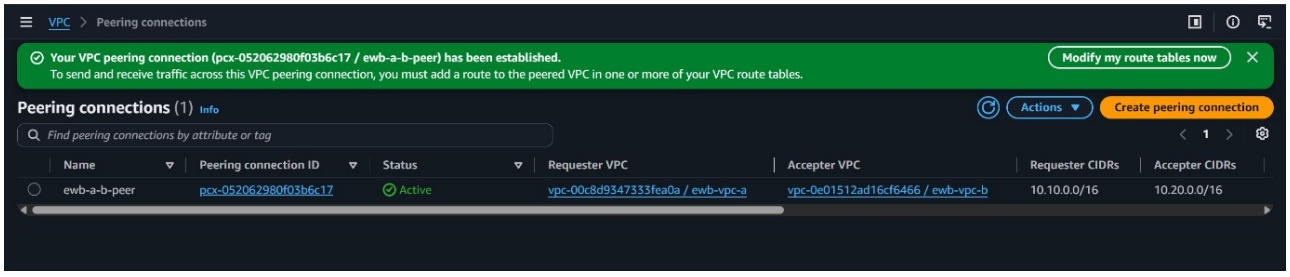


Figure 15. Peering connection ewb-a-b-peer is Active. Requester ewb-vpc-a (10.10.0.0/16), accepter ewb-vpc-b (10.20.0.0/16).

All rules use /32 sources, so only the lab hosts can connect. Nothing is open to 0.0.0.0/0.

Security group	Inbound rule	Source
ewb-sg-bastion	SSH / TCP 22	My public IPv4 ([REDACTED]/32)

Security group	Inbound rule	Source
ewb-sg-app-a	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-a	ICMP Echo request	10.20.1.10/32 (App-B)
ewb-sg-app-b	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-b	ICMP Echo request	10.10.2.10/32 (App-A)

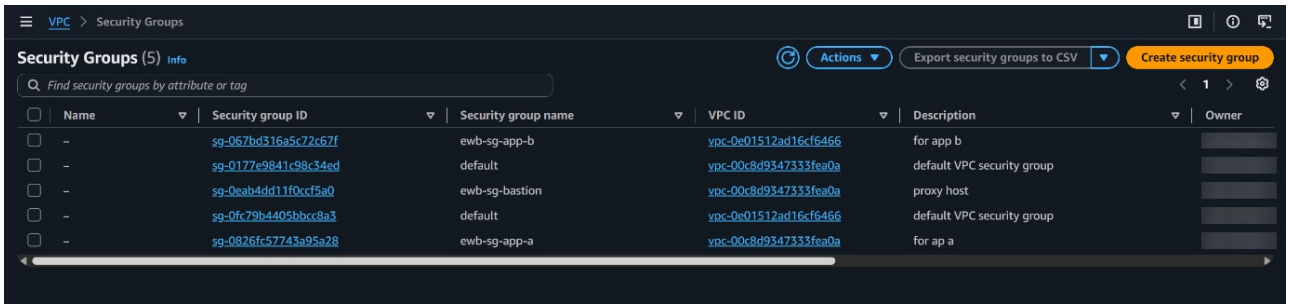


Figure 16. Security groups: ewb-sg-bastion (sg-0eab4dd11f0ccf5a0), ewb-sg-app-a (sg-0826fc57743a95a28), ewb-sg-app-b (sg-067bd316a5c72c67f).

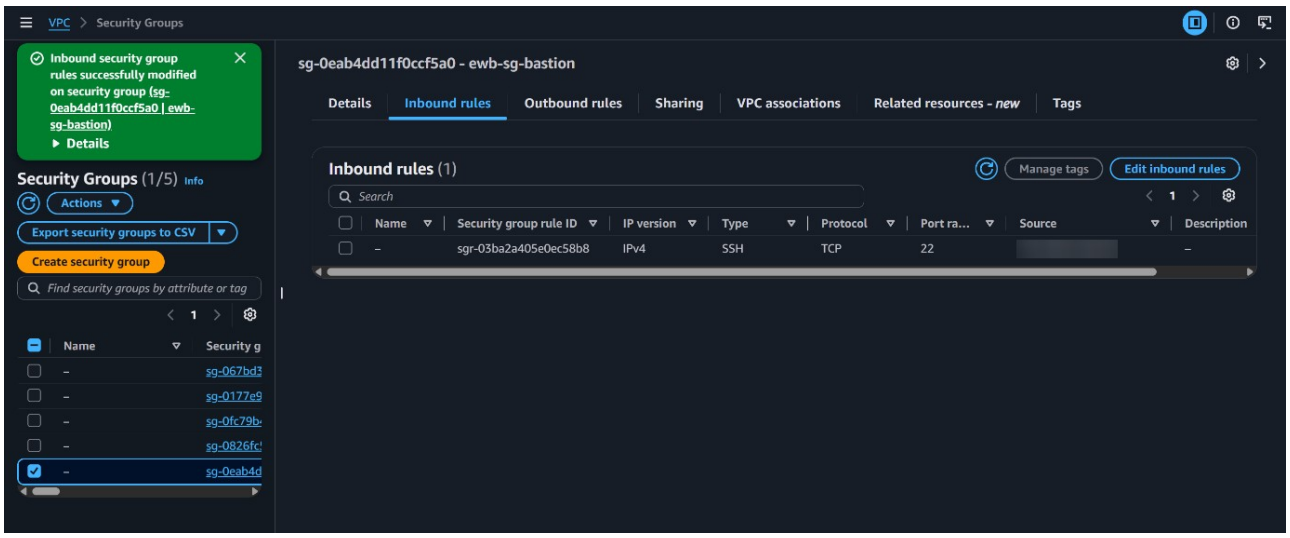


Figure 17. ewb-sg-bastion inbound rules: SSH 22 from my public IP /32.

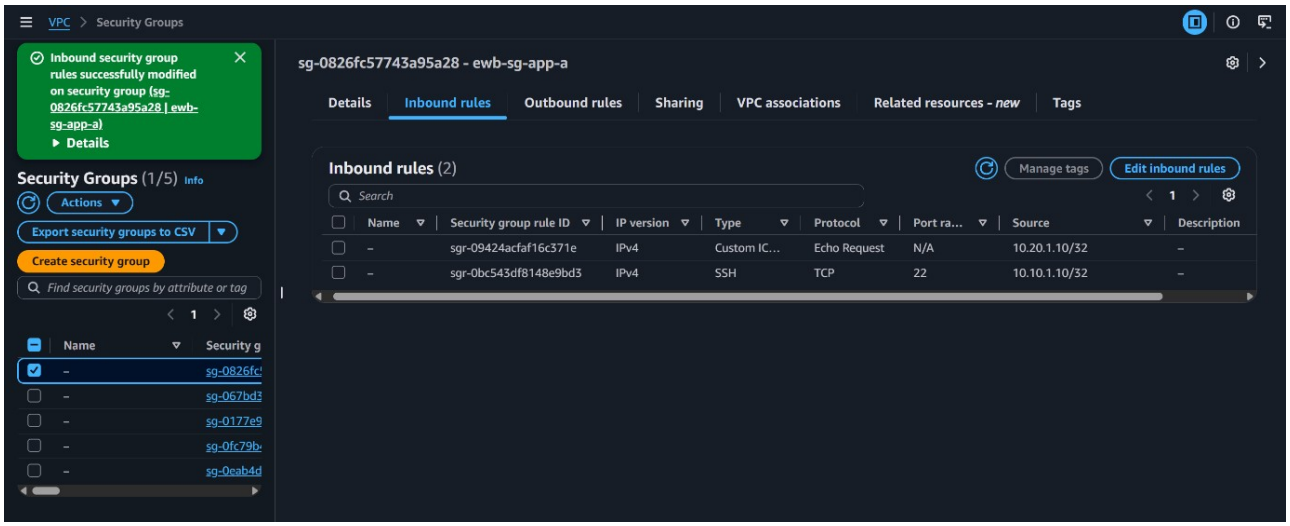


Figure 18. ewb-sg-app-a inbound rules: SSH 22 and ICMP Echo request from the bastion and App-B.

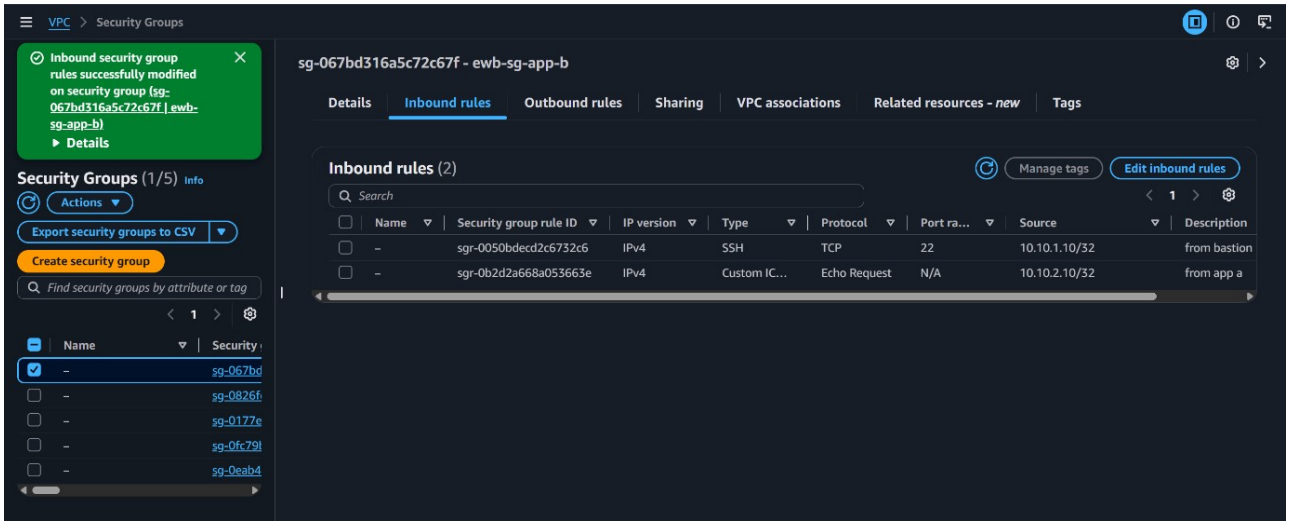


Figure 19. ewb-sg-app-b inbound rules: SSH 22 from the bastion and ICMP Echo request from App-A.

4. Test results

I ran every test myself and used private IPs only. All SSH commands ran on my laptop, using the bastion as ProxyJump so the private key never left my laptop.

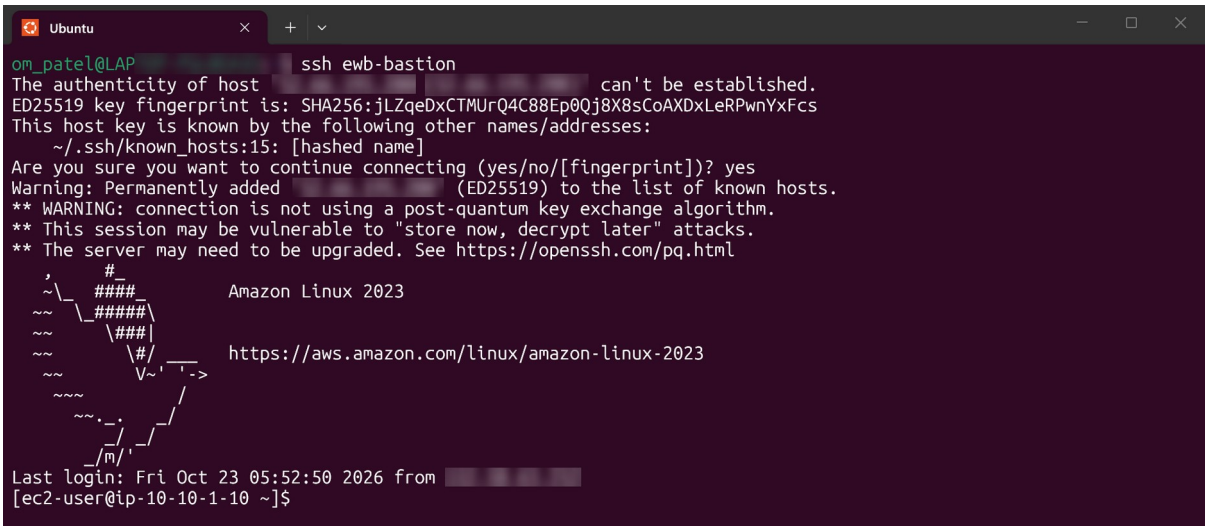


Figure 20. My laptop: ssh ewb-bastion connected to the bastion (ec2-user@ip-10-10-1-10).

Test	Where	Command	Expected	My result
01 Baseline (before peering)	App-A	ping -c 4 10.20.1.10	No replies	4 sent, 0 received, 100% loss
02 Forward path	App-A	ping -c 4 10.20.1.10	4 replies, 0% loss	4 received, 0% loss
03 Reverse path	App-B	ping -c 4 10.10.2.10	4 replies, 0% loss	4 received, 0% loss
04 Private SSH	My laptop	ssh ewb-app-b	Shell on App-B	Connected through the bastion
05 Isolation	EC2 console	Check public IPv4	None on App-A and App-B	Public IPv4 is empty on both



Figure 21. App-A, Test 01 baseline: ping to 10.20.1.10 before peering. Result: 100% packet loss.

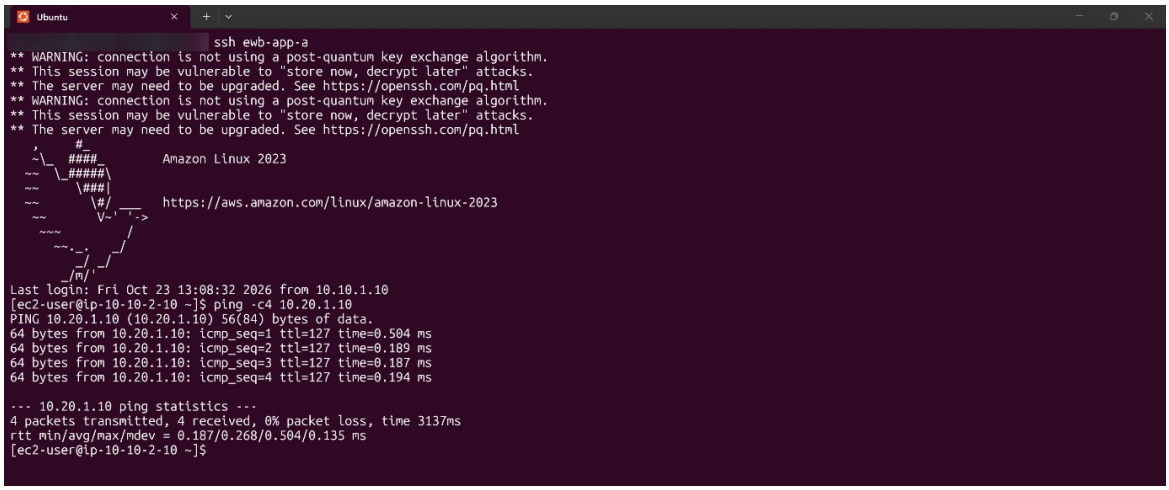


Figure 22. App-A, Test 02 forward path, first run after peering and routes: 4 received, 0% loss.

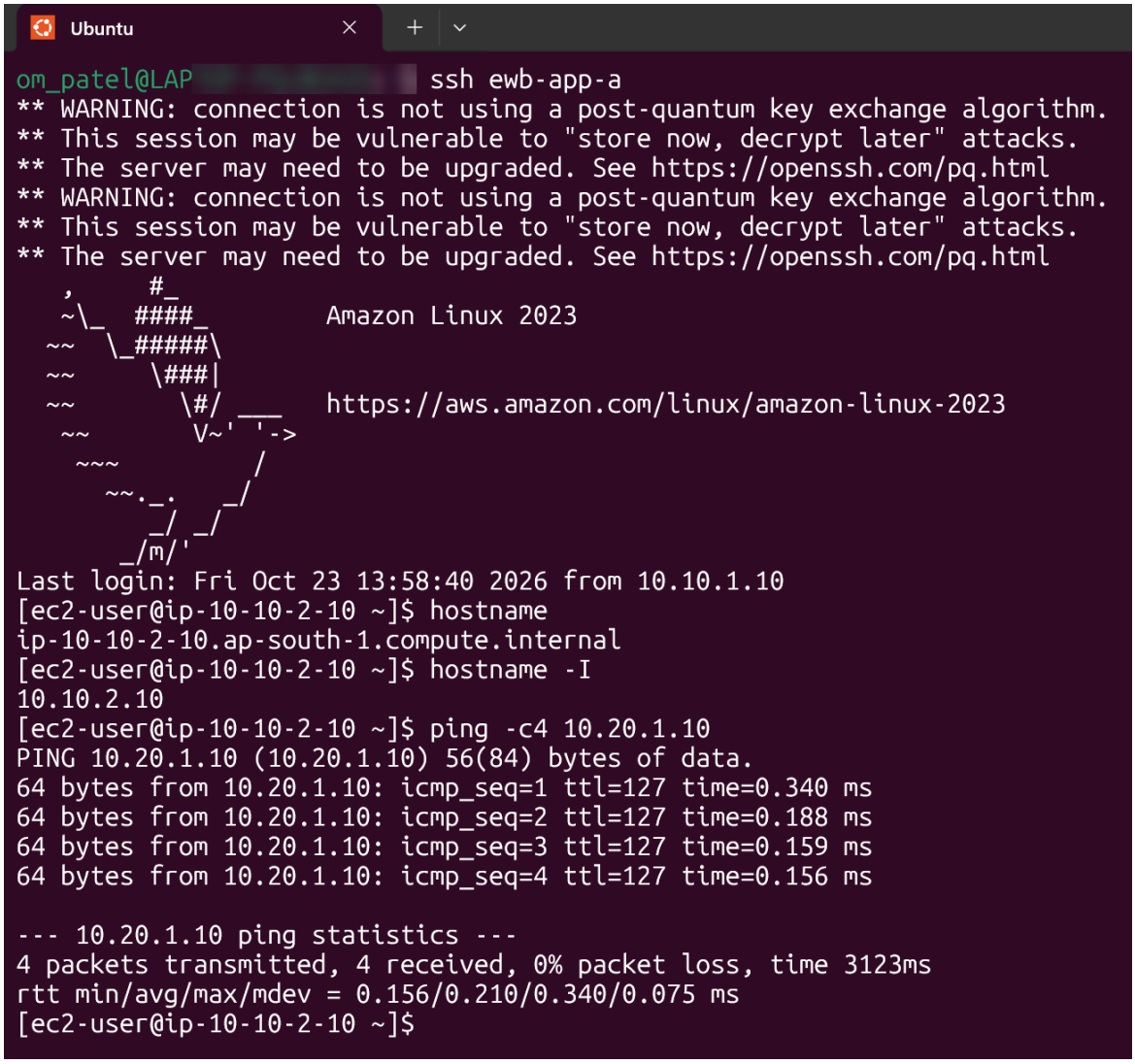


Figure 23. App-A, Test 02 forward path: hostname 10.10.2.10, ping to 10.20.1.10. Result: 4 received, 0% loss.

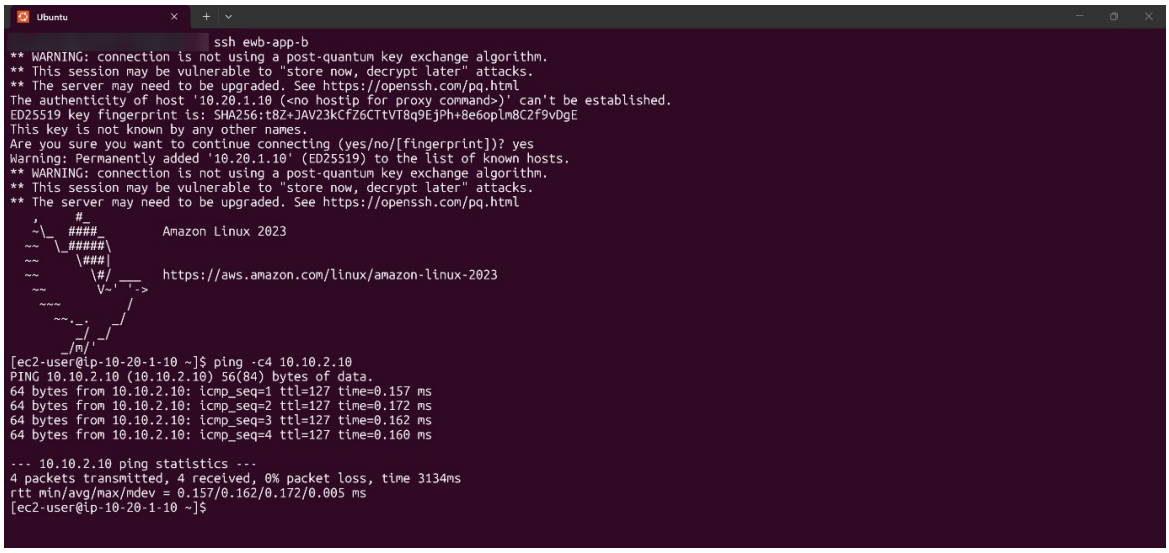


Figure 24. App-B, Test 03 reverse path, first run after peering and routes: ssh ewb-app-b, then ping to 10.10.2.10, 4 received, 0% loss.

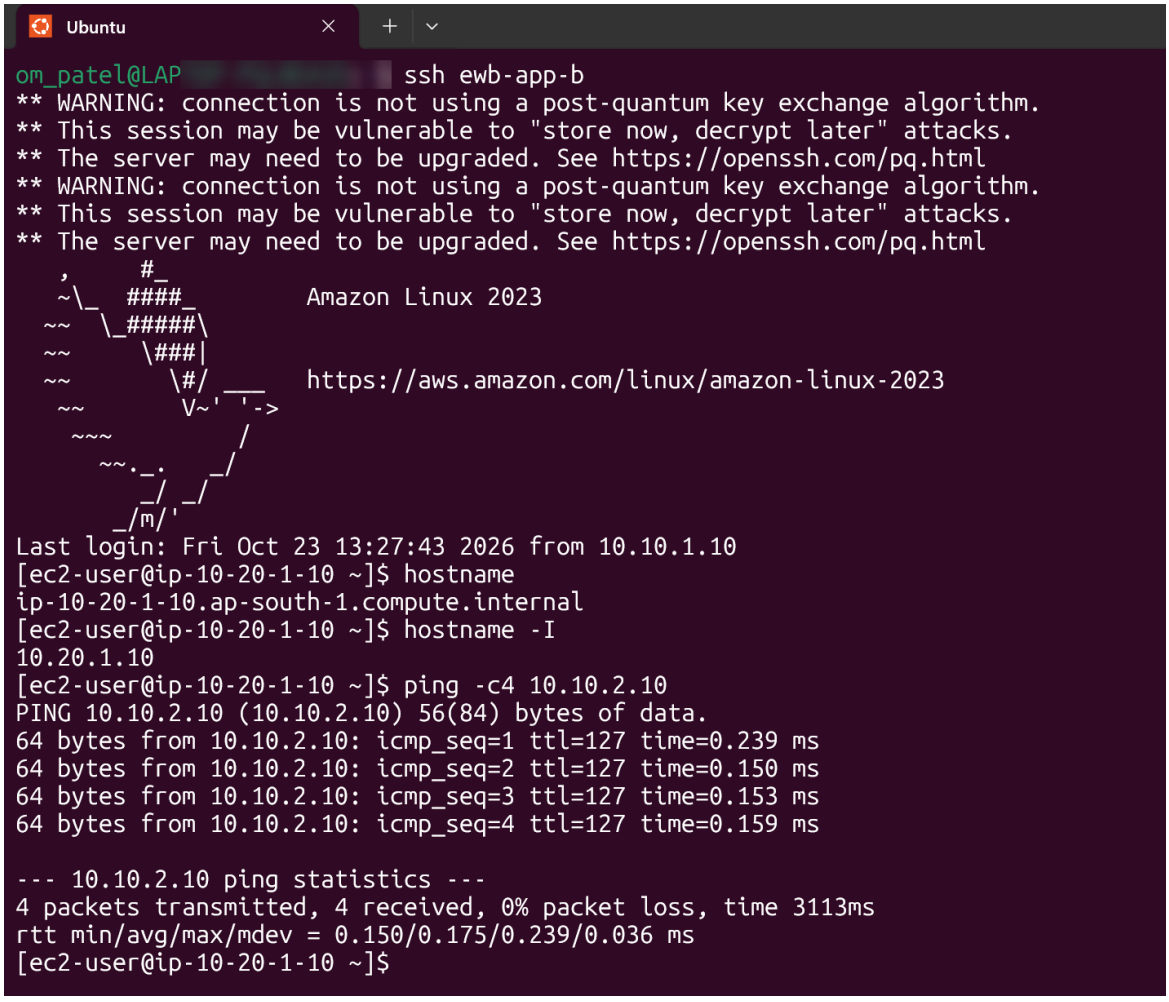


Figure 25. App-B, Test 03 reverse path: hostname 10.20.1.10, ping to 10.10.2.10. Result: 4 received, 0% loss.

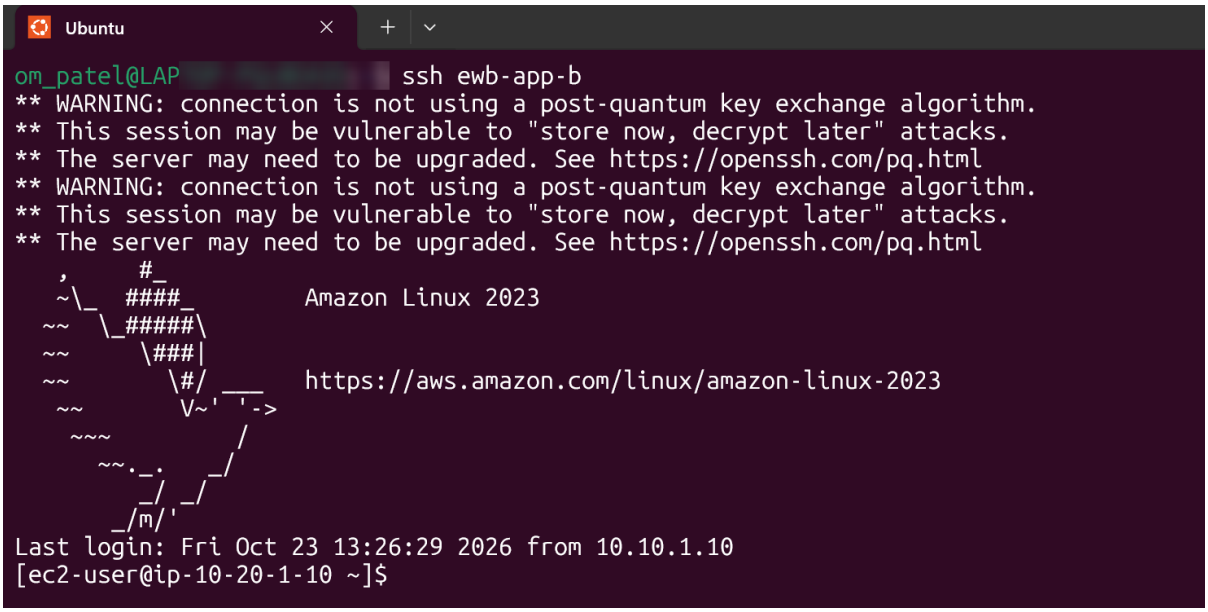


Figure 26. My laptop, Test 04 private SSH: ssh ewb-app-b reached App-B (ec2-user@ip-10-20-1-10) through the bastion.

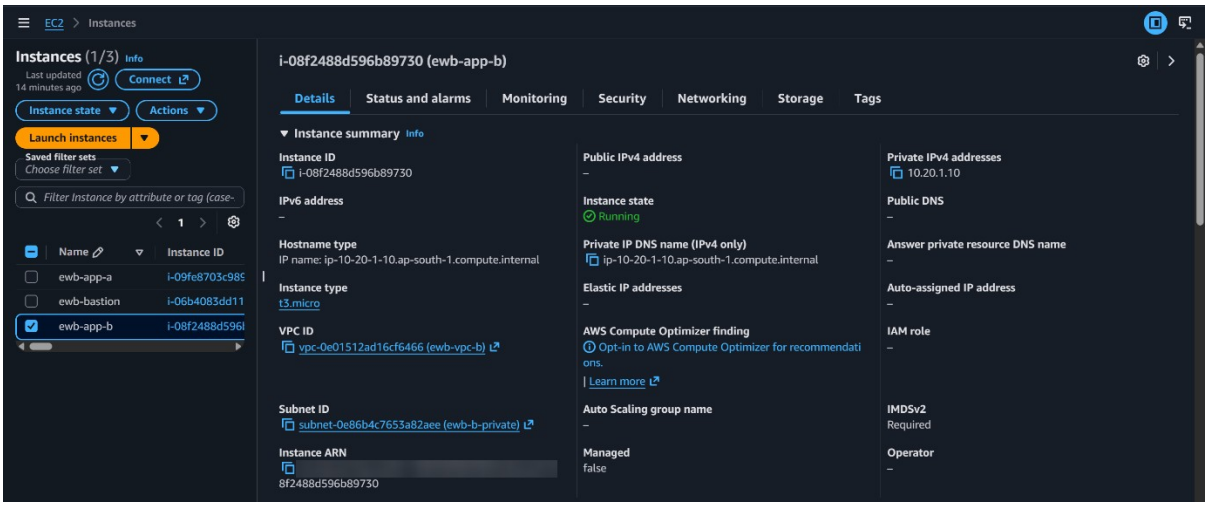


Figure 27. EC2 console, Test 05 isolation: ewb-app-b has private IP 10.20.1.10 and no public IPv4 address.

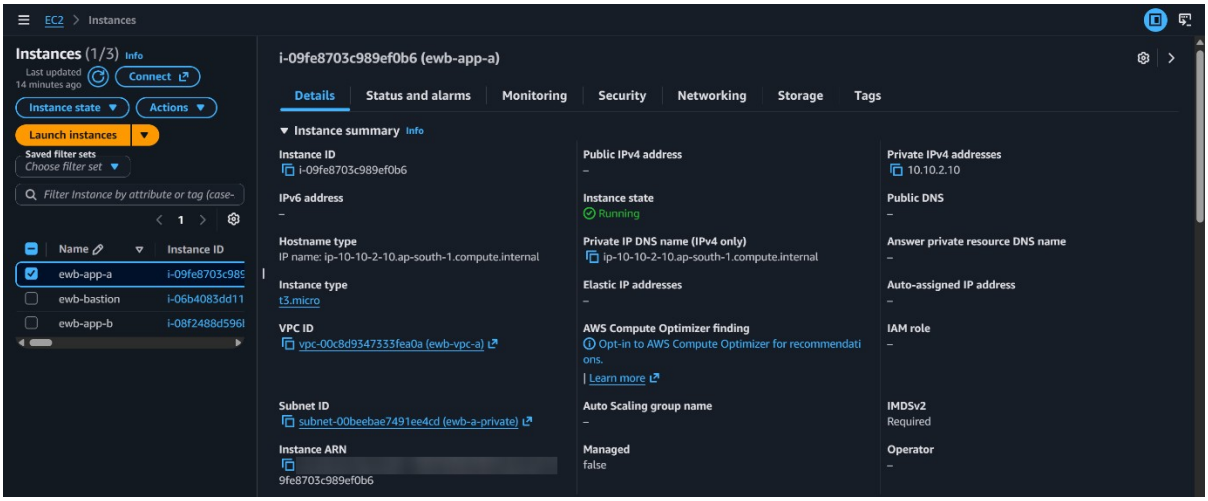


Figure 28. EC2 console, Test 05 isolation: ewb-app-a has private IP 10.10.2.10 and no public IPv4 address.

5. Controlled failure and recovery

I removed only the ICMP Echo request rule (from 10.10.2.10/32) on ewb-sg-app-b and pinged App-B from App-A again. I then added the exact same rule back and pinged again. The SSH rule was not touched.

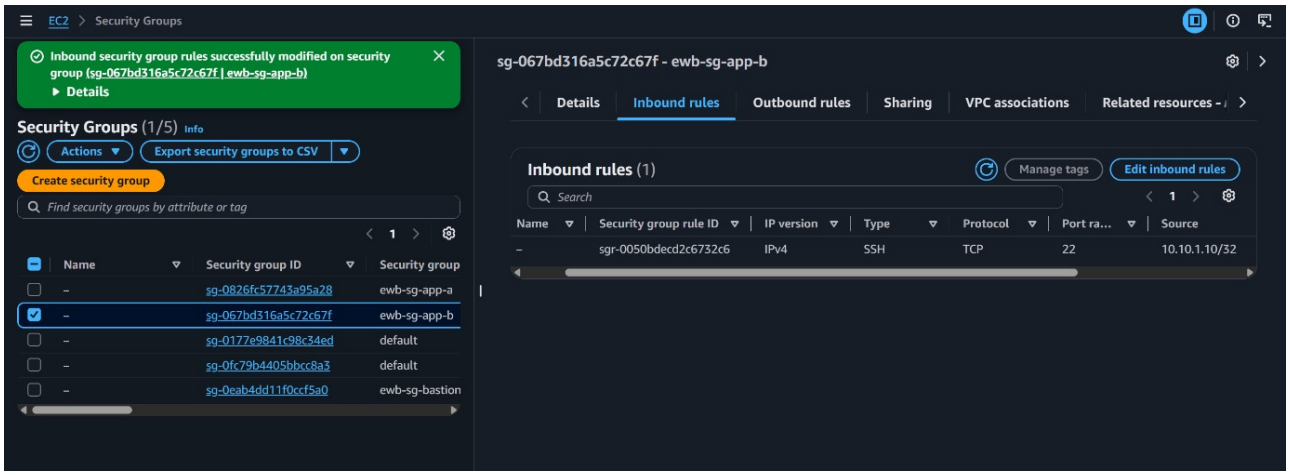


Figure 29. ewb-sg-app-b inbound rules after removing the ICMP rule: only SSH 22 from 10.10.1.10/32 is left.

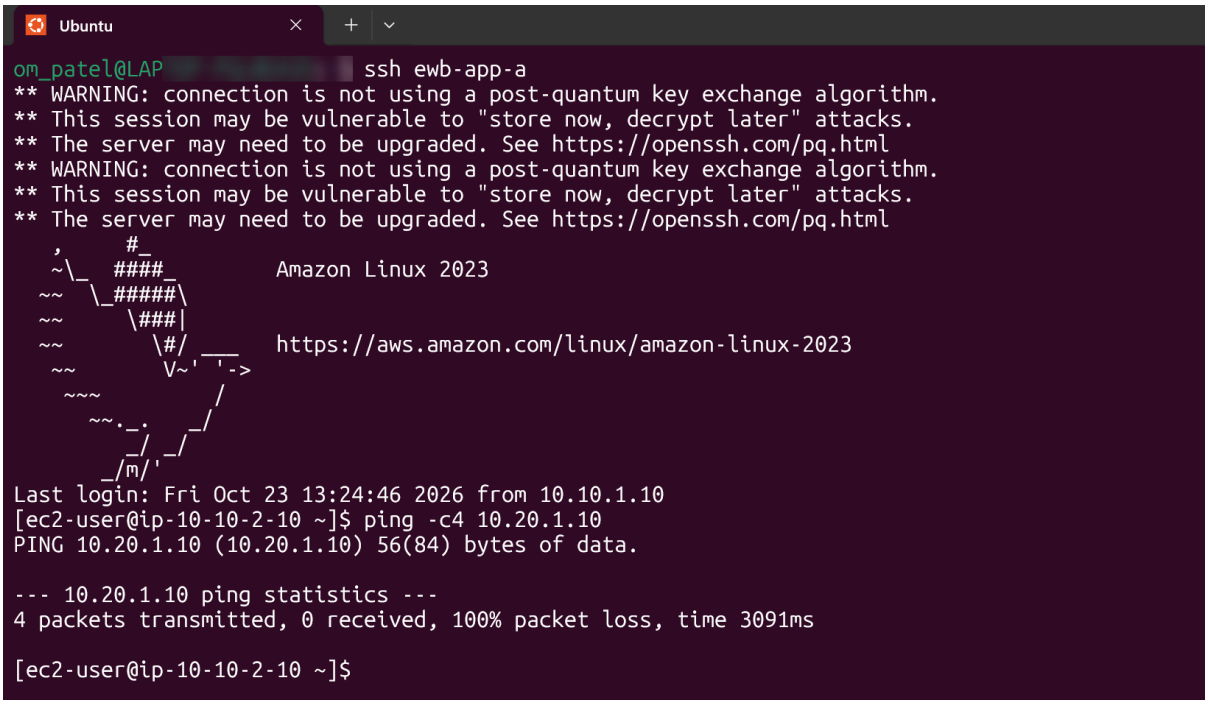


Figure 30. App-A: ping to 10.20.1.10 after the rule was removed. Result: 100% packet loss.

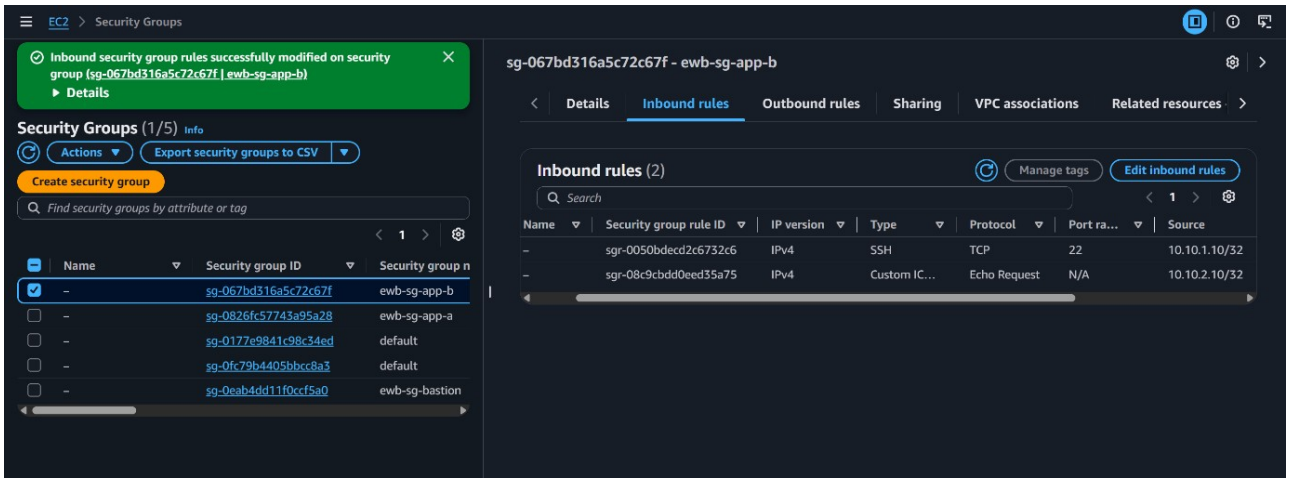


Figure 31. ewb-sg-app-b inbound rules after restoring the ICMP Echo request rule from 10.10.2.10/32.

```
Ubuntu
om_patel@LAP                    ssh ewb-app-a
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Fri Oct 23 13:24:46 2026 from 10.10.1.10
[ec2-user@ip-10-10-2-10 ~]$ ping -c4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3091ms

[ec2-user@ip-10-10-2-10 ~]$ ping -c4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.179 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.164 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.181 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.177 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3095ms
rtt min/avg/max/mdev = 0.164/0.175/0.181/0.006 ms
[ec2-user@ip-10-10-2-10 ~]$
```

Figure 32. App-A: same ping after the rule was restored. The first run failed, the second run received 4 replies with 0% loss.

What this shows: the route and the peering were still fine, but the security group on App-B dropped the pings. Routing and security groups are separate, and both must allow the traffic.

6. Forward path and return path

Forward path: App-A (10.10.2.10) sends a ping to 10.20.1.10. Its route table sends 10.20.0.0/16 to the peering connection, and the security group on App-B allows the ping, so it reaches App-B.

Return path: App-B replies to 10.10.2.10. Its route table sends 10.10.0.0/16 back through the peering connection, so the reply reaches App-A. Without this return route the reply would be lost and the ping would fail.

SSH to App-B: with ProxyJump the bastion opens its own connection to App-B, so App-B sees the bastion's private IP (10.10.1.10) as the source.

7. Cleanup

After saving my evidence I deleted the instances, volumes, peering routes, peering connection, security groups, subnets, route tables, internet gateway and both VPCs, and the key pair.

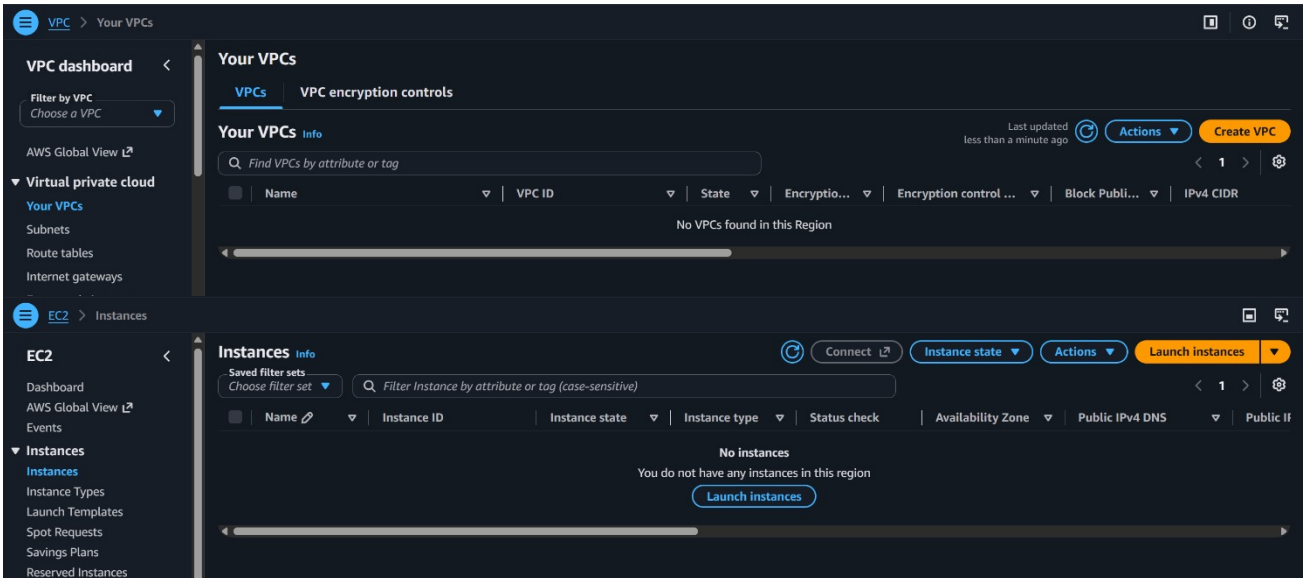


Figure 33. Cleanup evidence: EC2 and VPC consoles showing the lab resources deleted.

8. What I learned

1. Peering is only the link. Before I added routes, the peering connection was already Active, but the ping from App-A to App-B still failed with 100% packet loss. After I added the peering routes on both sides, the same ping got 4 replies with 0% loss. So I now understand that Active only means the connection exists. The subnets still need routes that point to it.

2. Routes are needed in both directions. App-A needs a route to 10.20.0.0/16 and App-B needs a route back to 10.10.0.0/16. If only one side has a route, the request arrives but the reply has no path back, so the ping or SSH still fails. The bastion's public subnet also needs the 10.20.0.0/16 route, because ProxyJump makes the bastion itself connect to App-B.

3. Security groups are a separate layer. In the controlled failure I removed only the ICMP Echo request rule on ewb-sg-app-b. The routes and the peering did not change, but the ping failed completely. After I added the same rule back, the ping worked again. This showed me that both the route table and the security group must allow the traffic, and that I should check both when something does not work.

4. Least-privilege rules. I used /32 sources for every rule: my own public IP for the bastion, and the private IPs of the bastion, App-A and App-B for the rest. Nothing is open to 0.0.0.0/0. App-B sees the bastion's private IP (10.10.1.10) as the SSH source, because the bastion opens its own connection when I use ProxyJump.

5. Private servers and the bastion. App-A and App-B have no public IPv4 address, and I confirmed this in the EC2 console. I still administered both from my laptop by using the bastion with ProxyJump, so the private key stayed on my laptop and was never copied to any instance. VPC B has no internet gateway, and peering does not give it internet access through VPC A.

6. Planning and cleanup. The two VPC ranges (10.10.0.0/16 and 10.20.0.0/16) do not overlap, which is required for peering. If they overlapped I would have had to rebuild a VPC. I also learned to troubleshoot in a fixed order: instance state, destination IP, route table, then security group. Finally, the lab does not need a NAT gateway, and deleting everything in the right order, then checking the project tag, avoids unexpected charges.